Functional Features & State Management Efficiency

1. Overview

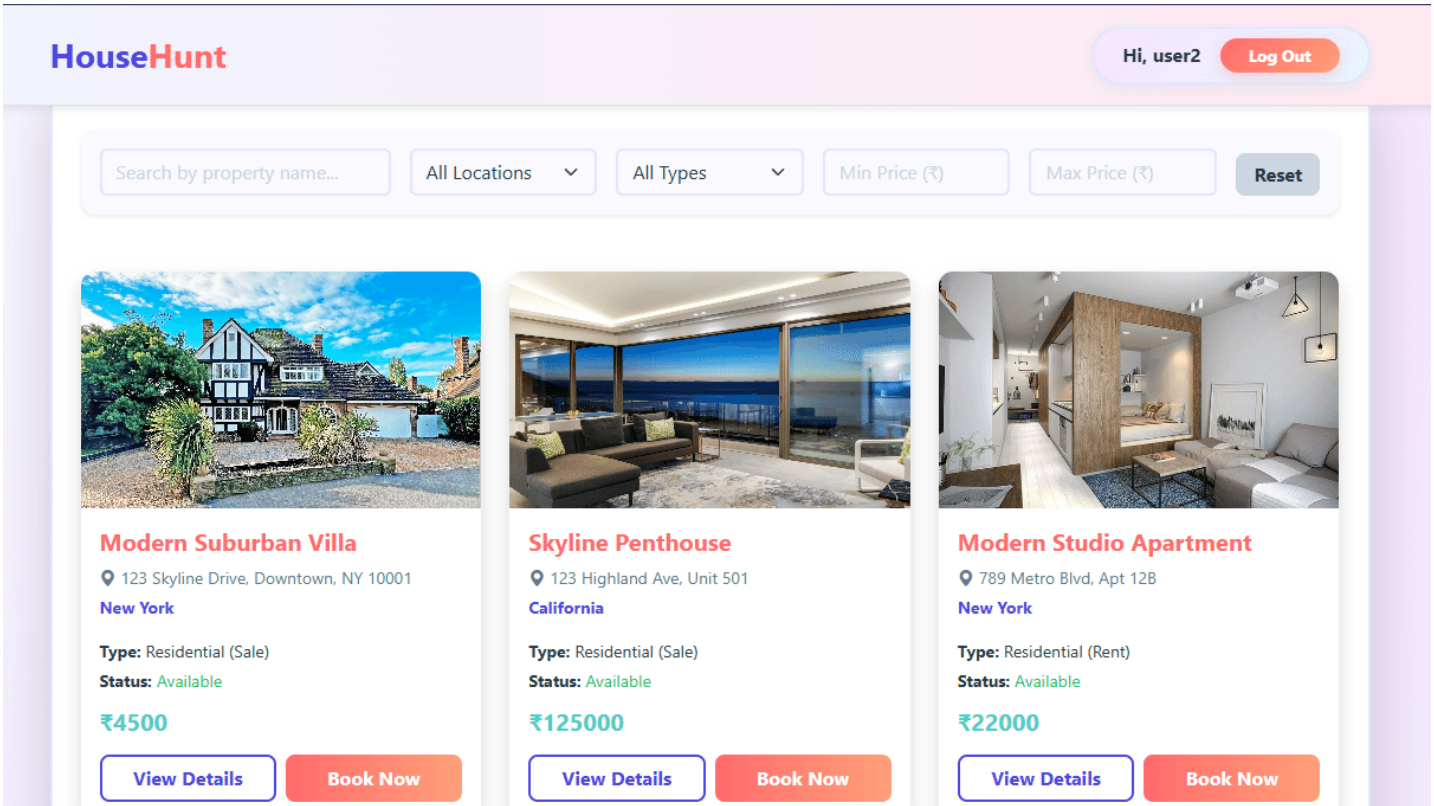
This document details the core functional features of the HouseHunt platform and the strategies employed to manage data state efficiently across both the React frontend and the Express/Node.js backend. Ensuring efficient state management was critical for maintaining real-time UI synchronization, preventing data corruption, and providing a seamless user experience.

---

2. Core Functional Features

The platform is driven by several highly interactive, role-specific functional modules:

- **Role-Based Dashboards:** Distinct interfaces with strict access control for Admins (global oversight), Owners (property management), and Renters (search and booking).
- **Dynamic Search & Filtering:** A responsive filtering engine allowing renters to sift through properties by State, Property Type (Commercial/Residential), and Price Range without triggering full-page reloads.



- **Booking Lifecycle Engine:** A complete end-to-end pipeline tracking requests from `pending` to either `booked` or `rejected`, with visual badges and dynamic button states reflecting the current status.
- **Integrated Wallet System:** A transactional feature allowing secure wallet deductions from the renter and additions to the owner's balance upon booking approval.

---

3. Frontend State Management (React)

To keep the application lightweight and responsive, we utilized a combination of React's built-in hooks and browser storage, avoiding the overhead of heavy external libraries (like Redux) where they were unnecessary.

3.1. Local & Component State (`useState` , `useEffect`)

- **Data Fetching:** Data such as `properties` and `myBookings` are fetched once upon component mount using `useEffect` and stored in local state arrays.
- **Controlled Inputs:** All search bars, filter dropdowns, and form inputs (like the Add Property form) are strictly controlled using `useState`, ensuring the UI always reflects the underlying data model.

3.2. Derived State for UI Efficiency

Instead of making separate API calls to check the booking status of *each* individual property, the platform uses **Derived State**.

- **Example:** In the Renter Dashboard, the system cross-references the `properties` array with the `myBookings` array in memory. It dynamically calculates the correct button UI (e.g., rendering "Booked (Yours)", "Request Pending", or "Not Available") on the fly. This drastically reduces server load and ensures instant UI rendering.

3.3. Session State (`localStorage` / `sessionStorage`)

- **Authentication:** JWT tokens, User IDs, Usernames, and Roles are securely stored in `localStorage` to maintain session persistence across page refreshes.
- **Flash Messaging:** Temporary success or error messages (e.g., "Property updated successfully!") are passed via `sessionStorage` and cleared upon rendering, providing clean user feedback.

---

4. Backend State Management & Data Integrity (Node/Express)

State efficiency on the server side is focused on transactional integrity and preventing conflicting data states in the MongoDB database.

4.1. Cascading State Updates

When a critical state change occurs, the backend automatically resolves dependent states.

- **The Double-Booking Block:** When an Owner approves a booking request (changing its state to `booked`), the backend controller executes a single transaction that:
  1. Deducts funds from the Renter's wallet.
  2. Adds funds to the Owner's wallet.
  3. Updates the Property status to `Unavailable`.
  4. *Automatically* bulk-updates all other `pending` requests for that specific property to `rejected`.

4.2. State Immutability

To preserve financial and historical records, certain states are programmed to be immutable. Once a booking is marked as `rejected` or `booked`, the backend actively blocks any further PUT/update requests to that specific document.

4.3. Orphaned Data Handling (Null State Fallbacks)

If an Owner completely deletes a property from the database, the old booking records attached to it remain in the Renter's history. Instead of allowing a `null` reference to crash the React frontend, the UI is programmed to detect this missing state and gracefully render a "Property Deleted" placeholder.

---

5. Conclusion

By leveraging derived state on the frontend to minimize API calls, and enforcing strict, cascading state updates on the backend, the HouseHunt platform achieves a high level of operational efficiency. The UI remains fast and responsive, while the underlying database maintains strict financial and transactional integrity.